Generating Keeper profiles for review

Anna Ostropolets

2026-03-26

Contents

1 Introduction **1**

2 Clinical Definition **1**

3 Creating Input Concept Sets **2**
3.1 Using LLMs to Generate Concept Sets 2
3.2 Manual Creation or Review of Concept Sets 3

4 Creating the Cohort to Evaluate **5**

5 Running Keeper **7**
5.1 Keeper Output 7

6 Reviewing the Keeper Profiles **8**
6.1 Review in a Spreadsheet 8
6.2 Review Using the Shiny App 8

1 Introduction

This vignette describes how to use Keeper to generate patient summaries for case adjudication from data mapped to the OMOP Common Data Model (CDM).

Keeper extracts and summarizes patient-level data for individuals in a specified cohort to facilitate the review of patient profiles. Such examinations can be used to interactively develop a phenotype (cohort definition) of a disease or—in its primary use case—to determine if patients truly have the disease and subsequently calculate positive predictive value (PPV).

This review should be conducted by someone familiar with the disease of interest, the underlying data, and the data collection process. Alternatively, the review can be performed by large language models (LLMs), which additionally enables the calculation of sensitivity and specificity. Please refer to the *Using Keeper with LLMs* vignette for more details on that use case.

2 Clinical Definition

The first step is to write a clinical definition. For this exercise, we will use a brief version, using Type 1 Diabetes Mellitus (T1DM) as our example phenotype.

> T1DM is an autoimmune condition characterized by decreased insulin production by the pancreas. Onset most commonly occurs in childhood or adolescence, but it can present in adults. Symptoms include weight loss, polyuria, polydipsia, fatigue, and others. Common differential diagnoses (conditions that must be ruled out) include type 2 diabetes, pancreatic disorders such as cystic

fibrosis, pancreatic necrosis, steroid-induced diabetes, renal glycosuria, and other conditions. Diagnostic procedures include glucose measurements, C-peptide, pancreatic and insulin antibodies, as well as HbA1c testing. It is primarily treated with insulin. Complications can include hypo- and hyperglycemia, neuropathy, nephropathy, cerebrovascular disease, and peripheral artery disease.

We will use this definition to construct our inputs. The concept of differential diagnosis is crucial; for each input category (except the disease of interest itself), we will consider concepts related to both T1DM and its differential diagnoses to evaluate evidence for the disease and to effectively rule out alternatives.

3 Creating Input Concept Sets

Keeper extracts data based on user-defined concept sets. If a concept belonging to an input concept set is found in a patient's records, Keeper will extract it along with its date relative to the index date. Therefore, careful concept set selection is highly important.

These input concept sets can be created manually, but this process can be challenging and labor-intensive. Alternatively, we can use large language models (LLMs) to generate initial concept sets.

3.1 Using LLMs to Generate Concept Sets

We use the `ellmer` package to connect to an LLM from your provider of choice, including Anthropic, Google, OpenAI, or a local LLM. For example, we can connect to OpenAI's ChatGPT using:

```
library(ellmer)
client <- chat_openai()
```

This assumes you have set the `OPENAI_API_KEY` environmental variable. See the ellmer package for details on connecting to various providers.

We also need access to a database containing the OHDSI Vocabulary tables. We specify the connection details like so:

```
library(DatabaseConnector)
connectionDetails <- createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/ohdsi",
  user = "joe",
  password = "supersecret"
)
vocabularyDatabaseSchema <- "cdm"
```

See the `createConnectionDetails()` documentation for more information on connecting to your database server.

Next, we can generate the concept sets:

```
conceptSets <- generateKeeperConceptSets(
  phenotype = "Type I Diabetes Mellitus (T1DM)",
  client = client,
  vocabConnectionDetails = connectionDetails,
  vocabDatabaseSchema = vocabDatabaseSchema
)
conceptSets
```

```
## # A tibble: 481 x 5
##    conceptId conceptName                                    vocabularyId conceptSetName       target
##        <int> <chr>                                          <chr>        <chr>                <chr>
##  1    201254 Type 1 diabetes mellitus                       SNOMED       doi                  Disease
```

```
## 2    435216 Disorder due to type 1 diabetes mellitus      SNOMED       doi                  Disease
## 3    201826 Type 2 diabetes mellitus                      SNOMED       alternativeDiagnosis Disease
## 4    195771 Secondary diabetes mellitus                   SNOMED       alternativeDiagnosis Disease
## 5    195212 Hypercortisolism                              SNOMED       alternativeDiagnosis Disease
## 6    438476 Vasopressin resistance                        SNOMED       alternativeDiagnosis Disease
## 7  40480068 Drug-induced hyperglycemia                    SNOMED       alternativeDiagnosis Disease
## 8   4163735 Hemochromatosis                               SNOMED       alternativeDiagnosis Disease
## 9    193170 Renal glycosuria                              SNOMED       alternativeDiagnosis Disease
## 10  37016349 Hyperglycemia due to type 2 diabetes mellitus SNOMED       alternativeDiagnosis Disease
## # i 471 more rows
```

3.2 Manual Creation or Review of Concept Sets

We can also create the concept sets manually, or, if we used an LLM to generate them, review the outputs. The format of the concept sets should be a data frame with the following columns:

- **`conceptId`**: The specific concept ID.
- **`conceptName`**: The name of the concept.
- **`vocabularyId`**: The vocabulary the concept belongs to.
- **`conceptSetName`**: The category of the concept set. Allowed values are: `"doi"`, `"alternativeDiagnosis"`, `"symptoms"`, `"drugs"`, `"diagnosticProcedures"`, `"measurements"`, `"treatmentProcedures"`, `"complications"`.
- **`target`**: Either `"doi"` or `"alternativeDiagnosis"`, depending on which condition the concept is related to. This distinction only matters for color-coding within the Shiny app.

Importantly, when using `useDescendants = TRUE` in `generateKeeper()` (which is the default setting), all descendants of the concepts specified here will automatically be included.

Below, we discuss each concept set category in detail.

3.2.1 DOI

DOI (Disease of Interest) is the target condition being evaluated. Here, we select two concepts along with their descendants:

- 201254 Type 1 diabetes mellitus
- 435216 Disorder due to type 1 diabetes mellitus

The first code represents T1DM itself, while the second code denotes diseases occurring due to T1DM, which implies the patient also has T1DM. A common strategy is to select the codes used as index event criteria in the phenotype definition. If `useAncestor` is set to `TRUE` (the default behavior), Keeper will use the hierarchy to pull in descendants of the selected concepts.

The DOI is looked up in the `CONDITION_OCCURRENCE` table.

3.2.2 Alternative Diagnosis

Alternative diagnoses are the competing conditions we want to rule out. Differential diagnoses for T1DM include the following conditions:

- 201826 Type 2 diabetes mellitus
- 4192640 Pancreatitis
- 4163735 Hemochromatosis

- 193170 Renal glycosuria
- ...

Alternative diagnosis codes are looked up in the `CONDITION_OCCURRENCE` table within 90 days before and after the index date.

Note: For all subsequent categories, we want to select the concepts relevant to the DOI *as well as those relevant to the alternative (competing/differential) diagnoses.*

3.2.3 Symptoms

Here we input symptoms typically occurring in T1DM and its differential diagnoses. These are signs and symptoms that occur in a short time window before disease onset.

Based on our clinical definition, we selected the following codes:

- 79936 Polyuria
- 432454 Excessive thirst
- 315078 Palpitations
- 439141 Abnormal weight gain

- 4134010 Weight decreased
- ...

These are broad SNOMED codes representing the symptoms we are interested in; source codes of the corresponding conditions map either to them directly or to their descendants.

A good approach for selecting codes for this section (and subsequent sections) is to input your term in ATLAS Search and click on the green shopping cart (the Phoebe initial code selection feature) to get a starting point. Then, use Phoebe (the Recommend tab within the ATLAS Concept Set module) to explore related recommendations. Instructions on how to use Phoebe can be found here. While you can explore your local data to find appropriate SNOMED codes using string searches, you are more likely to miss relevant codes this way.

Symptoms are looked up in the `OBSERVATION` and `CONDITION_OCCURRENCE` tables within the 30 days prior to the index date.

3.2.4 Drugs

We selected drugs (ancestor terms with their descendants) used to treat T1DM as well as the differential diagnoses:

- 1596977 insulin, regular, human
- 1502905 insulin glargine

- 503297 metformin
- 793143 semaglutide
- ...

Drugs are looked up in the `DRUG_ERA` table any time prior to and any time after the index date (displayed as two separate columns).

3.2.5 Diagnostic Procedures

Diagnostic procedures are the procedure codes used for diagnosing the disease of interest or alternative disease(s).

- 4083913 Urine specimen collection, 24 hours
- 4300757 Computed tomography
- ...

Diagnostic procedures are looked up in the `PROCEDURE_OCCURRENCE` table within 30 days prior to and after the index date.

3.2.6 Measurements

Measurements are laboratory tests used to diagnose T1DM and differential diagnoses:

- 4215993 Administration of insulin
- 3004410 Hemoglobin A1c/Hemoglobin.total in Blood
- 3007263 Hemoglobin A1c/Hemoglobin.total in Blood by calculation
- 40762352 Hemoglobin A1c/Hemoglobin.total in Blood by IFCC protocol
- 4073199 Total thyroidectomy
- ...

Note that there are often many variants of a given measurement. Be sure to include them all.

Measurements are looked up in the `MEASUREMENT` table within 30 days prior to and after the index date.

3.2.7 Treatment Procedures

Treatment procedures correspond to the treatment of the disease of interest or alternative disease(s). In this case, most procedures correspond to alternative diagnoses:

- 4215993 Administration of insulin

- 2000307 Unilateral adrenalectomy
- 4073199 Total thyroidectomy

Treatment procedures are looked up in the `PROCEDURE_OCCURRENCE` table any time after the index date.

3.2.8 Complications

Complications are other conditions occurring as a result of the disease. We selected the following codes along with their descendants:

- 24609 Hypoglycemia
- 4301699 Neuropathy
- 4262920 Skin ulcer
- 435517 Acidosis
- 4340390 Chronic hepatic failure
- ...

Complications are looked up in the `CONDITION_OCCURRENCE` table any time before or after the index date (displayed as two separate columns).

4 Creating the Cohort to Evaluate

Keeper creates profiles of persons in a specified cohort. Cohorts can be created using ATLAS, R, or SQL. The cohort table must contain the following fields:

- **`cohort_definition_id`** (INT): A unique identifier per cohort.
- **`subject_id`** (BIGINT): A unique identifier per person. This should correspond to the person ID in the CDM data.
- **`cohort_start_date`** (DATE): The date the person enters the cohort.
- **`cohort_end_date`** (DATE): (Optional) The date the person exits the cohort.

More information on creating cohorts can be found in the Book of OHDSI.

When using LLMs, it is also possible to create a highly sensitive cohort—a cohort that is unlikely to miss any true cases. Please refer to the *Using Keeper with LLMs* vignette for instructions on creating highly sensitive cohorts.

For this example, we will use the `Capr` package to define a simple cohort definition:

```r
library(Capr)

t1dmConceptIds <- c(201254, 435216)
t1dmCs <- cs(
  descendants(t1dmConceptIds),
  name = "Type 1 Diabetes Mellitus"
)

t1dmCohort <- cohort(
  entry = entry(
    conditionOccurrence(t1dmCs, firstOccurrence())
  ),
  exit = exit(
    endStrategy = observationExit()
  )
)
# Note: this will automatically assign cohort ID 1:
cohortSet <- makeCohortSet(t1dmCohort)
```

We can instantiate this cohort in our database using the `CohortGenerator` package. First, we must specify how to connect to the server holding the CDM data, and where the cohort table will be created:

```r
connectionDetails <- createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/ohdsi",
  user = "joe",
  password = "supersecret"
)
cdmDatabaseSchema <- "cdm"
cohortDatabaseSchema <- "cdm"
cohortTable <- "cohort"
options(sqlRenderTempEmulationSchema = NULL)
```

Next, we create the cohort table and generate the cohort:

```r
library(CohortGenerator)
connection <- connect(connectionDetails)
createCohortTables(
  connection = connection,
  cohortTableNames = getCohortTableNames(cohortTable),
  cohortDatabaseSchema = cohortDatabaseSchema
)
CohortGenerator::generateCohortSet(
  connection = connection,
  cdmDatabaseSchema = cdmDatabaseSchema,
  cohortDatabaseSchema = cohortDatabaseSchema,
  cohortTableNames = getCohortTableNames(cohortTable),
  cohortDefinitionSet = cohortSet
)
disconnect(connection)
```

5 Running Keeper

With a cohort and input concept sets defined, we are ready to run Keeper. We generate the Keeper profiles using the following code:

```
keeper <- generateKeeper(
  connection = connection,
  cohortDatabaseSchema = cohortDatabaseSchema,
  cdmDatabaseSchema = cdmDatabaseSchema,
  cohortTable = cohortTable,
  cohortDefinitionId = 1,
  sampleSize = 20,
  keeperConceptSets = conceptSets,
  phenotypeName = "Type I Diabetes Mellitus (T1DM)",
  removePii = TRUE
)
```

Here, `conceptSets` is the data frame we created earlier holding the target concepts. We use `cohortDefinitionId = 1` because that was implied in our code in the previous section. We specify a `sampleSize` of 20, meaning Keeper will randomly select up to 20 persons from the cohort. Alternatively, we could have provided a specific set of person IDs for Keeper to restrict its query to.

We provide a `phenotypeName` that will be stored with the Keeper profiles. We also set `removePii = TRUE` so that the output will not contain the original person IDs or absolute dates, ensuring the output is completely anonymized with no personally identifying information (PII).

5.1 Keeper Output

Keeper will populate the following categories with concepts observed in the person's records:

- **Patient ID:** (Optional, if `removePersonId = FALSE`).
- **Demographics:** Age, gender, race, and ethnicity.
- **Visit context:** Information about visits occurring in the window from 30 days prior to 30 days after the index date.
- **Observation period:** Information about overlapping `OBSERVATION_PERIOD` records, formatted as *days prior* to *days after* the index date.
- **Presentation:** All records in `CONDITION_OCCURRENCE` on day 0, along with their corresponding type and status.
- **Symptoms:** Records in `CONDITION_ERA` selected as symptoms within the 30 days prior to the index date, excluding day 0. This list does not include the disease of interest or complications. (If you want to track symptoms outside of this window, please place those codes in the complications concept set).
- **Prior disease:** Records in `CONDITION_ERA` selected as the disease of interest or complications at any time prior to the index date, excluding day 0.
- **Prior drugs:** Records in `DRUG_ERA` selected as drugs of interest at any time prior to the index date, excluding day 0, formatted as the day the era starts and the length of the drug era.
- **Prior treatment procedures:** Records in `PROCEDURE_OCCURRENCE` selected as treatments of interest at any time prior to the index date, excluding day 0.
- **Diagnostic procedures:** Records in `PROCEDURE_OCCURRENCE` selected as diagnostic procedures at any time prior to the index date, excluding day 0.
- **Measurements:** Records in `MEASUREMENT` selected as lab tests of interest within 30 days before and 30 days after day 0. These are formatted as value and unit (if available) and assessed against the reference range provided in the `MEASUREMENT` table (e.g., normal, abnormal high, abnormal low).
- **Alternative diagnosis:** Records in `CONDITION_ERA` selected as competing diagnoses within 90 days before and 90 days after day 0. This list does not include the disease of interest.
- **Post disease:** Same as prior disease, but occurring after day 0.
- **Post drugs:** Same as prior drugs, but occurring after day 0.

- **Post treatment procedures:** Same as prior treatment procedures, but occurring after day 0.
- **Death:** Any death record occurring after day 0.

6 Reviewing the Keeper Profiles

We can review the Keeper profiles manually or use an LLM, as described in the *Using Keeper with LLMs* vignette. When reviewing profiles manually, we can choose to do this in a spreadsheet program like Microsoft Excel or by using the built-in Shiny app.

6.1 Review in a Spreadsheet

We can convert the output of `generateKeeper` into a data frame having one row per person, with one column for each of the Keeper output categories:

```
keeperTable <- convertKeeperToTable(keeper)
keeperTable
```

```
## # A tibble: 20 x 21
##    generatedId cohortPrevalence phenotype age   sex    observationPeriod      race  ethnicity presen
##          <dbl>            <dbl> <chr>     <chr> <chr>  <chr>                  <chr> <chr>     <chr>
##  1          16           0.0488 T1DM      67    MALE   -6 days - 725 days     ""    ""        Corona
##  2           8           0.0488 T1DM      70    FEMALE -231 days - 507 days   ""    ""        Old my
##  3           6           0.0488 T1DM      72    MALE   -175 days - 1838 days  ""    ""        Type 1
##  4          14           0.0488 T1DM      61    MALE   -473 days - 256 days   ""    ""        Type 1
##  5          10           0.0488 T1DM      68    MALE   -336 days - 5832 days  ""    ""        Disord
##  6           5           0.0488 T1DM      78    MALE   -86 days - 2483 days   ""    ""        Type 1
##  7           7           0.0488 T1DM      70    FEMALE -2451 days - 1936 days ""    ""        Type 1
##  8          15           0.0488 T1DM      81    MALE   -1317 days - 2022 days ""    ""        Type 1
##  9           9           0.0488 T1DM      79    FEMALE -402 days - 1419 days  ""    ""        Type 1
## 10          13           0.0488 T1DM      79    MALE   -4179 days - 1274 days ""    ""        Disord
## 11          17           0.0488 T1DM      84    MALE   -201 days - 533 days   ""    ""        Disord
## 12          18           0.0488 T1DM      81    MALE   -111 days - 1722 days  ""    ""        Type 1
## 13          19           0.0488 T1DM      68    MALE   -380 days - 363 days   ""    ""        Type 1
## 14           3           0.0488 T1DM      81    FEMALE -566 days - 1362 days  ""    ""        Renal
## 15           1           0.0488 T1DM      74    FEMALE -44 days - 1063 days   ""    ""        Ketoac
## 16           2           0.0488 T1DM      78    MALE   -398 days - 2520 days  ""    ""        Disord
## 17          12           0.0488 T1DM      70    FEMALE -1453 days - 715 days  ""    ""        Type 1
## 18          11           0.0488 T1DM      69    FEMALE -5 days - 1709 days    ""    ""        Type 1
## 19          20           0.0488 T1DM      72    FEMALE -315 days - 782 days   ""    ""        Type 1
## 20           4           0.0488 T1DM      73    MALE   -69 days - 662 days    ""    ""        Type 1
## # i abbreviated names: 1: priorTreatmentProcedures, 2: postTreatmentProcedures
## # i 4 more variables: alternativeDiagnoses <chr>, diagnosticProcedures <chr>, measurements <chr>, dea
```

We can then save this table to a CSV file and open it in Excel:

```
readr::write_csv(keeperTable, "e:/temp/KeeperT1dm.csv")
```

6.2 Review Using the Shiny App

Alternatively, we can launch the built-in Shiny app to review the profiles interactively:

```
launchReviewerApp(
  keeper = keeper,
  keeperConceptSets = conceptSets,
  decisionsFileName = "decisions.csv"
)
```

This will launch the Shiny application, which looks like this:

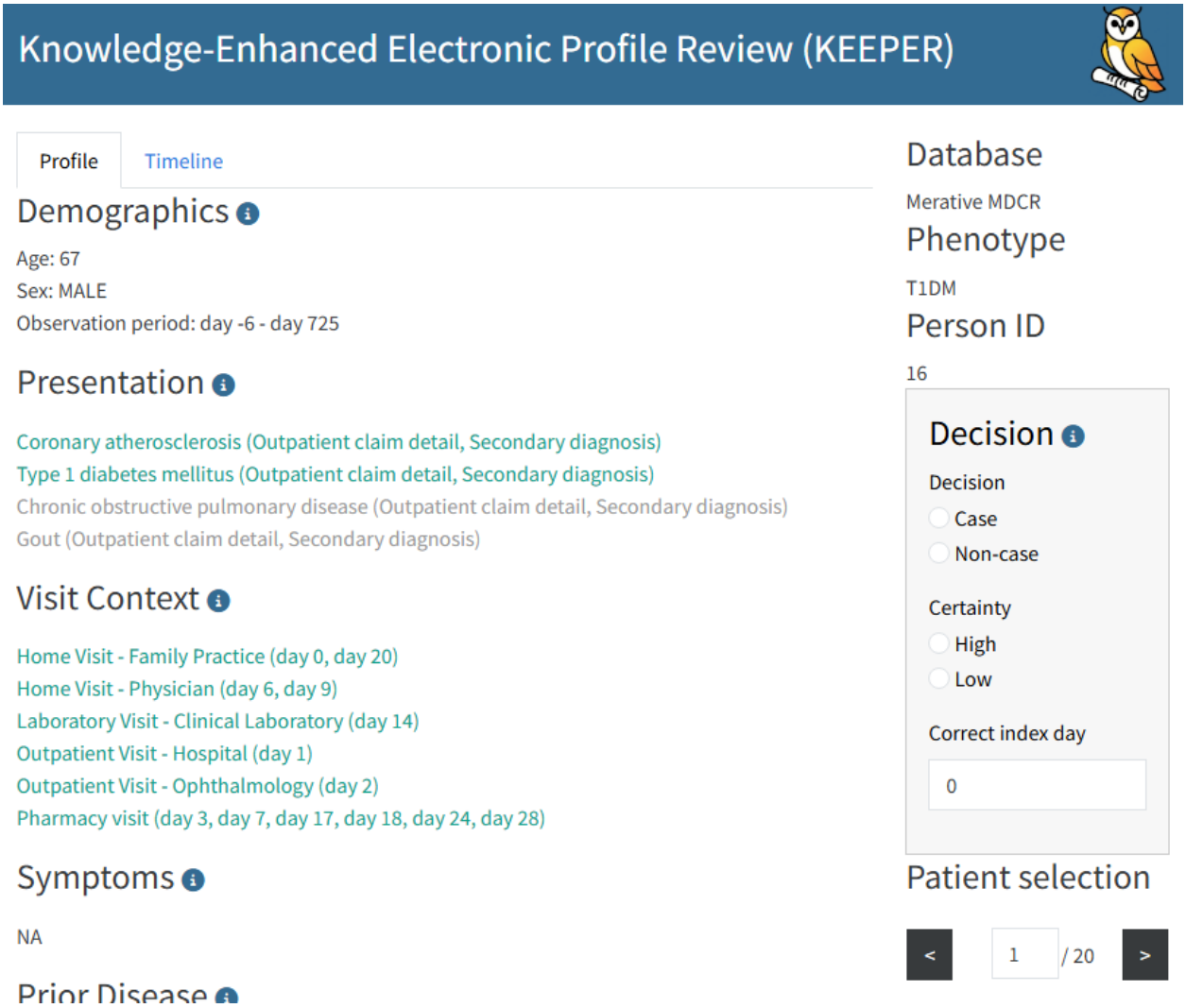

Figure 1: The Keeper Shiny app

Any decisions you log within the app will be written to the specified decisions file (e.g., `decisions.csv`). If the decisions file does not yet exist, Keeper will create it for you.